Applied Statistical Programming: Problem Set 3

Bowen Damask

2026-11-11

%All you need to do is knit this to pdf and it will render the tex file. As always, let me know if something isnt working or rendering correctly. I also used "High performance functions with Rcpp - by Hadley Wickham, the rccp lecture notes, and AI to help brainstorm some of my answers. All errors are my own, and I am more than happy to provide my written work as well.

Question 1.

Write a function in R and one in C++, that takes an integer n and returns TRUE if it is even and FALSE otherwise. Test both functions using $n \in \{4, 7, 10, 15\}$.

```
x <- c(4, 7, 10, 15)

evennumber <- function(n) n %% 2 == 0

cppFunction('bool evennumbercpp(int n) {return n % 2 == 0; }')

data.frame(x, r = sapply(x, evennumber), cpp = sapply(x, evennumbercpp))
```

```
##    x     r   cpp
## 1  4  TRUE  TRUE
## 2  7 FALSE FALSE
## 3 10  TRUE  TRUE
## 4 15 FALSE FALSE
```

Question 2.

Write an R function called ***stepR(n)*** *that implements this function using* ***if*** *and* ***else****. Write the equivalent C++ function called* ***stepCpp(n)****. Test both functions for* $n \in \{-3, 0, 3, 5, 10\}$.

```
x <- c(-3, 0, 3, 5, 10)

stepR <- function(n) {if (n < 0) {0} else if (n < 5) {1} else {2}}

cppFunction('int stepCpp(double n) {if (n < 0) return 0;
    if (n < 5) return 1;
    return 2;}')

data.frame(x, r= sapply(x, stepR), cpp = sapply(x, stepCpp))
```

```
##    x r cpp
## 1 -3 0   0
## 2  0 1   1
## 3  3 1   1
## 4  5 2   2
## 5 10 2   2
```

Question 3.

Write an R function called ***piR(N)*** *that generates* N *random points and estimates* π*. Use a* ***for*** *loop. Write the equivalent C++ function* ***piCpp(N)*** *using a* ***for*** *loop. Calculate the estimate for* $N \in \{100, 1{,}000, 10{,}000, 100{,}000\}$*. Compare the accuracy of the estimates as* N *increases.*

```
set.seed(21523514)
N <- c(100, 1000, 10000, 100000)

piR <- function(N) {
  inside <- 0
  for (i in seq_len(N)) inside <- inside + (runif(1)^2 + runif(1)^2 <= 1)
  4 * inside / N
}

cppFunction('double piCpp(int N) {
  int inside = 0;
  for (int i = 0; i < N; i++) {
    double x = R::runif(0, 1), y = R::runif(0, 1);
    inside += x * x + y * y <= 1;
  }
  return 4.0 * inside / N;
}')

data.frame(N, estimater = sapply(N, piR), estimatecpp = sapply(N, piCpp))
```

```
##       N estimater estimatecpp
## 1 1e+02    3.4000     3.04000
## 2 1e+03    3.1200     3.24800
```

```
## 3 1e+04    3.1408     3.14880
## 4 1e+05    3.1396     3.14064
```

Overall, regardless of whether done in r or in c++, we observe that as the N gets larger, the estimates for both the R code and the cpp code start to converge to approx. π. In general, we observe that as N grows larger, the accuracy of the estimate increases/error shrinks, though since the draws are random it does not have to improve at every step (as seen in the R estimate where its not exactly a strictly linear convergence). We also observe a slight difference between the R and cpp estimates, which is just because they are using different random draws.

Question 4.

Write the equivalent C++ function called `multiplyCpp(A, B)` using two nested `while` loops. Verify that both functions return the matrix shown above. Compare the execution times of your R and Rcpp functions using `microbenchmark()`. Compare the results with R's built-in matrix multiplication operator `%*%`. Which implementation is fastest?*

```
set.seed(21523514)

A <- matrix(rnorm(10000), 100)
B <- matrix(rnorm(10000), 100)

multiplyr <- function(A, B) {
  C <- matrix(0, nrow(A), ncol(B))
  i <- 1
  while (i <= nrow(A)) {
    j <- 1
    while (j <= ncol(B)) { C[i, j] <- sum(A[i, ] * B[, j]); j <- j + 1 }
    i <- i + 1
  }
  C
}

cppFunction('NumericMatrix multiplycpp(NumericMatrix A, NumericMatrix B) {
  NumericMatrix C(A.nrow(), B.ncol());
  int i = 0;
  while (i < A.nrow()) {
    int j = 0;
    while (j < B.ncol()) { C(i, j) = sum(A(i, _) * B(_, j)); j++; }
    i++;
  }
  return C;
}')

all.equal(multiplyr(A, B), A %*% B)
```

```
## [1] TRUE
```

```
all.equal(multiplycpp(A, B), A %*% B)
```

```
## [1] TRUE
```

```
microbenchmark(multiplyr(A, B), multiplycpp(A, B), A %*% B)
```

```
## Warning in microbenchmark(multiplyr(A, B), multiplycpp(A, B), A %*% B): less
## accurate nanosecond times to avoid potential integer overflows
```

```
## Unit: microseconds
##               expr       min        lq      mean    median        uq       max
##    multiplyr(A, B) 21139.928 21482.012 28001.629 21822.844 35959.808 40558.676
##  multiplycpp(A, B)  2216.747  2221.155  2240.039  2228.698  2242.659  2690.502
##            A %*% B   482.447   483.513   487.456   485.604   490.401   515.698
##  neval cld
##    100 a
##    100  b
##    100   c
```

In general, both the R and the cpp functions give the same matrix as `A %*% B`. Using microbenchmark() we see that `%*%` is by far the fastest to run, at about 0.4 ms. The next fastest is `multiplyCpp()` at about 12 ms, which makes sense when you consider its running without a translation layer, and then `multiplyR()` is the slowest at about 19 ms since that is running with the R translation layer. So the cpp code is faster than the R code. The `%*%` beats both since it is designed to do matrix multiplication in a very fast way :).

Question 5.

The first few values are: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55. Write a recursive R function called ***`fibR(n)`*** *that computes the n-th Fibonacci number. Write an equivalent C++ function called* ***`fibCpp(n)`****. Verify that* ***`fibR()`*** *and* ***`fibCpp()`*** *both produce the same result for* $n \in \{10, 20, 30\}$*. Why does the recursive implementation become increasingly slow as n increases? Does C++ solve this computational problem?*

```
x <- c(10, 20, 30)

fibr <- function(x) if (x < 2) x else fibr(x - 1) + fibr(x - 2)

cppFunction('int fibcpp(int x) { return x < 2 ? x : fibcpp(x - 1) + fibcpp(x - 2); }')

data.frame(x, r = sapply(x, fibr), cpp = sapply(x, fibcpp))
```

```
##    x      r    cpp
## 1 10     55     55
## 2 20   6765   6765
## 3 30 832040 832040
```

```
system.time(fibr(30))
```

```
##    user  system elapsed
##   0.664   0.012   0.676
```

```
system.time(fibcpp(30))
```

```
##    user  system elapsed
##   0.003   0.000   0.003
```

So in general the recursive implementation becomes increasingly slow as we increase n because numbers consecutively get called in a way which means the same values just keep getting calculated over and over. This means the number of calls we have to do multiplies by approx. 3/2 every time n increases by 1, so the time grows exponentially in n. For 100 calls it would be something crazy like over 100,000 years using R. C++ doesnt really solve this computational problem, it makes it faster and the calls cheaper because your running it without translation, but eventually the issue will still occur.